The Journal of Systems and Software 228 (2025) 112455

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Human-understandable explanation for software vulnerability prediction☆

Hong Quy Nguyen [a,*], Thong Hoang [b], Hoa Khanh Dam [a], Guoxin Su [a], Zhenchang Xing [b], Qinghua Lu [b], Jiamou Sun [b]

[a] University of Wollongong, 2 Northfields Avenue, Keiraville, 2500, NSW, Australia
[b] CSIRO Data61, Level 5/13 Garden St, Eveleigh, 2015, NSW, Australia

ARTICLE INFO

Keywords:
Vulnerability prediction
Explainable AI
Text generation
Key aspects

ABSTRACT

Recent advances in deep learning have significantly improved the performance of software vulnerability prediction (SVP). To enhance trustworthiness, the SVP highlights predicted lines of code (LoC) that may be vulnerable. However, providing LoC alone is often insufficient for software practitioners, as it lacks detailed information about the nature of the vulnerability. This paper introduces a novel framework that is built on SVP by offering additional explanatory information based on the suggested LoC. Similar to security reports, our framework comprehensively explains the vulnerability aspects, such as Root Cause, Impact, Attack Vector, and Vulnerability Type. The proposed framework is powered by transformer architectures. Specifically, we leverage pre-trained language models for code to fine-tune on two practical datasets: BigVul and Vulnerability Key Aspect, ensuring our framework's applicability to real-world scenarios. Experiments using the ROUGE and BLEU scores as evaluation metrics show that our framework achieves better performance with CodeT5+, statistically outperforming a baseline study in generating key vulnerability aspects. Additionally, we conducted a small-scale user study with experienced software practitioners to assess the effectiveness of the framework. The results show that 72% of the participants found our framework helpful in accepting the SVP results, and 68% rated the additional explanations as moderately to extremely useful.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

1. Introduction

Software vulnerabilities are inherent weaknesses or flaws that can arise during the software development process (Liu et al., 2012; Krsul, 1998; Ghaffarian and Shahriari, 2017). Recently, software vulnerabilities have become increasingly prevalent, leading to significant repercussions. For example, SolarWinds vulnerability caused considerable damage to numerous government bodies and enterprises (Jibilian and Canales, 2021; Wang et al., 2023a). Similarly, Log4Shell, a vulnerability associated with Apache Log4j, affected countless infrastructures of major firms, including Amazon, Cloudflare, and Tencent, resulting in severe consequences (Hunter and Vynck, 2021). Given these events, the identification and mitigation of software vulnerabilities play an important part in the software development lifecycle.

As software increases significantly in size and complexity, identifying software vulnerabilities becomes more complex and time-consuming. To address this challenge, developers predominantly employ two traditional approaches: static and dynamic analysis (Kuang et al., 2006; Aggarwal and Jalote, 2006; Kim et al., 2016). Static analysis involves examining the code without running it. It is a cost-effective method since it can identify bugs early during code review and development. Techniques such as data flow analysis (Sampaio and Garcia, 2016), control flow analysis (Sparks et al., 2007), and symbolic execution (Avgerinos et al., 2014) are instrumental for this purpose. In contrast, dynamic analysis requires the execution of software to reveal potential vulnerabilities, using methods such as fuzz testing (Sutton et al., 2007), taint analysis (James Newsome, 2005), and penetration testing (Shah and Mehtre, 2015). However, static and dynamic techniques can produce high false-positive rates (Aloraini et al., 2019), thus undermining their benefits (Smith et al., 2015).

The emergence of deep learning (DL) has opened new avenues to improve vulnerability prediction. Recent trends in vulnerability research have led to the use of techniques such as natural language understanding (Russell et al., 2018; Dam et al., 2018a; Fu and Tantithamthavorn, 2022), graph-based methods (Li et al., 2021; Chakraborty et al.,

☆ Editor: Alexander Serebrenik.

* Corresponding author.
E-mail addresses: nguyen.hquy@gmail.com (H.Q. Nguyen), james.hoang@data61.csiro.au (T. Hoang), hoa@uow.edu.au (H.K. Dam), guoxin@uow.edu.au (G. Su), zhenchang.xing@data61.csiro.au (Z. Xing), qinghua.lu@data61.csiro.au (Q. Lu), frank.Sun@data61.csiro.au (J. Sun).

https://doi.org/10.1016/j.jss.2025.112455
Received 27 October 2024; Received in revised form 12 March 2025; Accepted 7 April 2025
Available online 26 April 2025
0164-1212/Crown Copyright © 2025 Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

2022), and hybrid models (Hin et al., 2022). Despite the high accuracy of DL models in predicting vulnerabilities, their nature as a "black-box" leads to limited explainability. This issue particularly hinders the adoption of DL models in software development processes for two reasons. First, human verification is essential in any process to mitigate the potential risks associated with unwarranted decision-making. Therefore, in the context of software vulnerability prediction (SVP), explainability is equally important as precision (Dam et al., 2018b). Second, with the volume of daily code changes, knowing the impact of the predictions allows developers to prioritize their debugging efforts.

Explainable artificial intelligence (XAI) has emerged as a pivotal research domain with the aim of elucidating deep learning decision-making processes. However, research on XAI for vulnerability prediction remains relatively sparse compared to the anticipation of the broader community. In terms of vulnerability prediction, XAI techniques are frequently used to suggest specific localized explanations, e.g., lines of code (LoC) that might cause vulnerabilities in software systems (Fan et al., 2020; Fu and Tantithamthavorn, 2022). However, that kind of explanation is too rough to grasp, as it does not include an overview of vulnerability, such as what the type is and how vulnerability would be impacted. Thus, it is hard to prioritize efforts as well.

Recent studies have shown that providing explanations in a mental model, such as human language, is a promising approach to increasing trust in vulnerability prediction (Hoffman et al., 2018; Achtibat et al., 2022). However, an adequate mental model specifically designed for vulnerability prediction remains absent. Therefore, high-level, vague, and untrustworthy explanations hinder the adoption of vulnerability prediction in software development processes.

In order to fill the gap, we follow the current security community practice to facilitate human understanding of software vulnerabilities. The current practice is that security experts use text (natural language) to refer to the multidimensional aspects of a vulnerability; they are *Root Cause, Impact, Attack Vector, and Vulnerability Type.* Those aspects are generated from predicted LoC from a SVP method, thus, our practical implication on the software field is to make explanations given to software practitioners more intuitive and accessible.

The novelty of this study lies in the design of a model that enhances software vulnerability prediction (SVP) by leveraging pre-trained language models to generate human-understandable explanations from lines of code (LoC). Our contribution begins with an approach to enrich dataset by merging and processing data from BigVul (Fan et al., 2020) and Vulnerability Key Aspects (VKA) (Sun et al., 2023), which provide real-world C/C++ vulnerability data and security reports, respectively. Next, our study involves a systematic comparison of factors influencing explanation quality, including the selection of the optimal pre-trained model and the configuration of input LoC. Furthermore, we conduct a user study to validate the practical utility of our generated explanations for software developers during SVP result verification. To the best of our knowledge, this is the first study to specifically address the generation of human-understandable explanations for SVP.

The package of the dataset, code, trained models, and other artifacts is available at https://github.com/quy-ng/human-xai-software-vulnerability-prediction

2. Motivating example

Consider the **CVE-2024-0222** vulnerability in Listing 1, which is from Google's Chrome project. This vulnerability was identified in 2024, and contributions to fixing it attracted a bounty award of $15,000.[1] This example illustrates the result of state-of-the-art vulnerability prediction and line explanation. We apply LineVul (Fu and Tantithamthavorn, 2022), which is currently considered a prominent approach to explaining vulnerability prediction. LineVul predicts that the **copyImage()** function in Listing 1 is a vulnerability. Suppose that we want LineVul to return the top three code lines, which are the key contributors to its prediction, then it outputs line #8, #9, and #10. The vulnerability actually starts from variable **src** at line #2, then **src** flows to line #10. The patch[2] for this vulnerability is to impose a logic check to verify whether variable **src** is a copy of the same texture that prevents Chrome from flushing data to GPU.[3]

Fixing a vulnerability is hard, even for experienced developers. If developers do not know about a vulnerability, they may think the flag is a false alert, and then they can merge the code into the production codebase. Thus, overlooking vulnerabilities can lead to persistence in a live production environment for days, posing significant and potentially catastrophic risks. Although finding an LoC that likely contains a vulnerability is useful, the predicted LoC provides little information about the nature of the vulnerability. As in practical security reports, developers need to know the root cause of a vulnerability and the exploitation technique that can be used to exploit it. In addition, LoC does not tell developers the type of vulnerability and the possible impact. Hence, LoC does not help developers prioritize their focus among daily tasks based on vulnerability impacts.

```
1  angle::Result TextureVk::copyImage(const gl
       ::Context *context, const gl::
       ImageIndex &idx,
2    const gl::Rectangle &sourceArea, GLenum
         interFrt, gl::Framebuffer *src)
3  {
4    RendererVk *renderer = vk::GetImpl(
         context)->getRenderer();
5    gl::Extents newImageSize(sourceArea.width
         , sourceArea.height, 1);
6    const gl::InternalFormat &interInfo =
7          gl::GetInternalFormatInfo(interFrt,
               GL_UNSIGNED_BYTE);
8    const vk::Format &vkFrt = renderer-
     >getFormat(interInfo.sizedInternalFormat);
9    ANGLE_TRY(redefineLevel(context, idx, vkFrt,
     newImageSize));
10   return copySubImageImpl(context,
     idx, gl::Offset(0, 0, 0), sourceArea, interInfo,
     src);
11 }
```

Listing 1: A code vulnerability, identified as CVE-2024-0222, is detected by the LineVul algorithm Fu and Tantithamthavorn (2022) as an vulnerable code. We highlight the top three code lines that contribute to this vulnerability. We note that we rename some variables to fit with our paper visualization

In the security community, a software vulnerability is described in different key aspects, including: (1) the **Attack Vector**, which delineates the exploitation technique; (2) the **Root Cause**, highlighting the preconditions leading to a vulnerability; (3) the **Impact**, indicating the potential consequences of a given weakness; and (4) the **Vulnerability Type**, which offer an explanation to possible function vulnerabilities. These four aspects are widely used to describe a vulnerability in the security community. In fact, they are from the CVE specification (Anon, 2018). A CVE description has generic human-readable aspects that form

[1] https://chromereleases.googleblog.com/2024/01/stable-channel-update-for-desktop.html

[2] https://chromium.googlesource.com/angle/angle/+/3f6ffdb286bdfcb8be570714201dca6e1c9f2027

[3] https://issues.chromium.org/issues/40942112

a template as follows: "**[Vulnerability Type]** in [Component] in [Vendor] [Product] [Version] allows [Attacker] to **[Impact]** via **[Vector]**" and [COMPONENT] in [Vendor] [Product] [Version]"**[Root Cause]** allows [Attacker] to **[Impact]** via **[Vector]**".[4]

CVE-2024-0222's specification has a technical version published on the Chromium issue tracker,[3] and a general version published on the NVD website.[5] From NVD website, Vulnerability Type is "use-after-free", and Attack Vector is "a crafted HTML page". From the issue tracker, security experts, who find the vulnerability, mention Impact: "attackers have GPU executed unwanted commands from an HTML page". In addition, the experts explain the Root Cause as "an invalid context, that is currently attached to the framebuffer for rendering, bypass WebGL's check". Our study aims to provide such information from predicted LoC.[6]

3. Approach

3.1. Framework overview

Our study is based on our observation that once an SVP method provides predicted a vulnerable LoC as its explanation, because those lines likely contain vulnerability context. In addition, security analysts tend to focus on a few recommended lines (Parnin and Orso, 2011) rather than the whole code function content. Therefore, our study uses the k-top predicted LoC as our input. Given a k-top predicted LoC, our objective is to generate human-understandable text from the input. This leads us to the usage of the sequence-to-sequence (Seq2seq) strategy, which is widely used in generating natural language processing tasks. We address different types of human-understandable explanations by opting multi aspects, which are vital to construct a security report. However, there is a lack of datasets that consist of the relation between vulnerable lines (or predicted lines) and key vulnerability aspects. Thus, in our study, we build new datasets for that purpose. Without loss of generality, we select LineVul (Fu and Tantithamthavorn, 2022), a state-of-the-art prediction method that can predict the top k vulnerable LoC in a code function, as a representative SVP method.

Fig. 1 provides an overview of our proposed framework to provide human-understandable aspect explanations for software vulnerability predictions. Similar to deep learning approaches, our framework consists of two phases, namely training and inference. In both phases, our framework needs the out of LineVul (Fu and Tantithamthavorn, 2022), which is the top k vulnerable LoC, as our input. The framework's outputs are explanations in terms of the four key vulnerability aspects: Attack Vector, Root Cause, Impact, and Vulnerability Type.

In the training phase, the first task is data acquisition and pre-processing. The data is collected from two sources, BigVul (Fan et al., 2020) and Vulnerability Key Aspects (Sun et al., 2023). As illustrated in Fig. 1, we create four datasets corresponding to the four key aspects after merging and separating. The data structure of the data points in each dataset is identical: pairs of code functions and their explanation reference. In addition, each dataset is divided into three subsets for training, validation, and testing (refer to Section 3.2.3 for data size). In the next task, we learn four sequence-to-sequence (Seq2seq) transformer models, each targeting a dataset. The learning process follows: the code function is fed into LineVul to get the k-predicted vulnerable lines. Then, those lines are inputted into the corresponding Seq2seq model to generate an explanation text. Later, the generated explanation text is compared with the explanation reference to minimize training loss.

In the inference phase, we deploy four trained Seq2seq models to generate explanations for the top k predicted LoC from LineVul. Therefore, the output of our framework is a set of four key aspects, which are used to describe the k-vulnerable LoC.

3.2. Data acquisition and pre-processing

Our approach utilizes two prominent software vulnerability datasets, namely *BigVul* and *Vulnerability Key Aspects* (*VKA*).

3.2.1. BigVul

This dataset is an important asset for vulnerability research; it contains a collection of C/C++ code functions categorized into two classes: non-vulnerable and vulnerable (Fan et al., 2020), and tagging CVE to vulnerable functions. It includes 188,636 code functions, with a 5.7% incidence rate of vulnerable code functions, spanning CVE records from 2002 through 2019. The dataset includes contributions from influential, large-scale projects like Chrome,[7] Linux,[8] Android,[9] and the PHP interpreter,[10] each of which holds significant sway across diverse tech communities and retains its relevance in the current technological landscape. In addition, BigVul has been widely used in recent studies and state-of-the-art approaches for vulnerability prediction (Hin et al., 2022; Wang et al., 2023b; Fu and Tantithamthavorn, 2022; Li et al., 2021). More importantly, LineVul is trained and evaluated on the BigVul dataset, which is why we chose BigVul as our dataset. We emphasize that our approach can be extended to other datasets if a vulnerability prediction method is trained on those datasets.

3.2.2. Vulnerability Key Aspects (VKA)

A security report typically exhibits a multi-faceted (in other words, multi-aspects) structure, encompassing key aspects such as Vulnerability Type, Root Cause, Attack Vector, and Impact. However, it is not mandatory to present these aspects simultaneously. We demonstrate this phenomenon in Fig. 2, where the CVE-2024-0222 report contains Vulnerability Type, Impact, and Attack Vector, while CVE-2024-1086 lacks Attack Vector but includes Root Cause. Additionally, the report is often written in free text, posing processing challenges. Consequently, we adopt the VKA dataset from Sun et al. (2023), which is a map from a CVE to its aspects. The authors of VKA initially employed entity recognition and question–answer models to extract key aspects from vulnerability reports. Subsequently, they manually verify the distilled data to ensure its accuracy. Furthermore, VKA utilizes two databases, the National Vulnerability Database (NVD) (Anon, 2022a) and IBM's X-Force (Anon, 2022b), to address missing aspects. For instance, if a report in NVD lacks any aspects, the authors attempt to identify the missing aspects in IBM's X-Force. Despite their efforts, not all CVEs contain all four aspects; only 19% of CVEs encompass all four aspects. Nevertheless, to our knowledge, VKA is the best dataset for our study.

In general, VKA consists of reports from 1999 to 2020, contains 146,807 CVEs, and covers more than five programming languages such as C/C++, PHP, Java, Python, JavaScript, and others. However, according to the published numbers from VKA, C/C++ vulnerabilities represent 38.1% of the dataset.

3.2.3. Data pre-processing

This section details the process of merging the BigVul and VKA datasets (illustrated in Fig. 1). We aim to create four separate datasets for each key aspect of vulnerability. This is because given a Vulnerability Type, Impact, Attack Vector, and Root Cause can be completely different. For example, in Fig. 2, both CVE-2024-0222 and CVE-2024-1086 have the same Vulnerability Type, which is "use-after-free", but Impacts are different. Moreover, since BigVul covers only C/C++ language, we opt out of other languages in VKA. For each key aspect, our procedure is as follows:

[4] Since we do not target at a specific Vendor, Product, Component, and Version, those aspects are not included in our study.

[5] https://nvd.nist.gov/vuln/detail/CVE-2024-0222

[6] We will report the output of the motivation example in Section 4.3.1.

[7] https://www.google.com/intl/en_au/chrome/

[8] https://www.linux.org/

[9] https://developer.android.com/

[10] https://onlinephp.io/

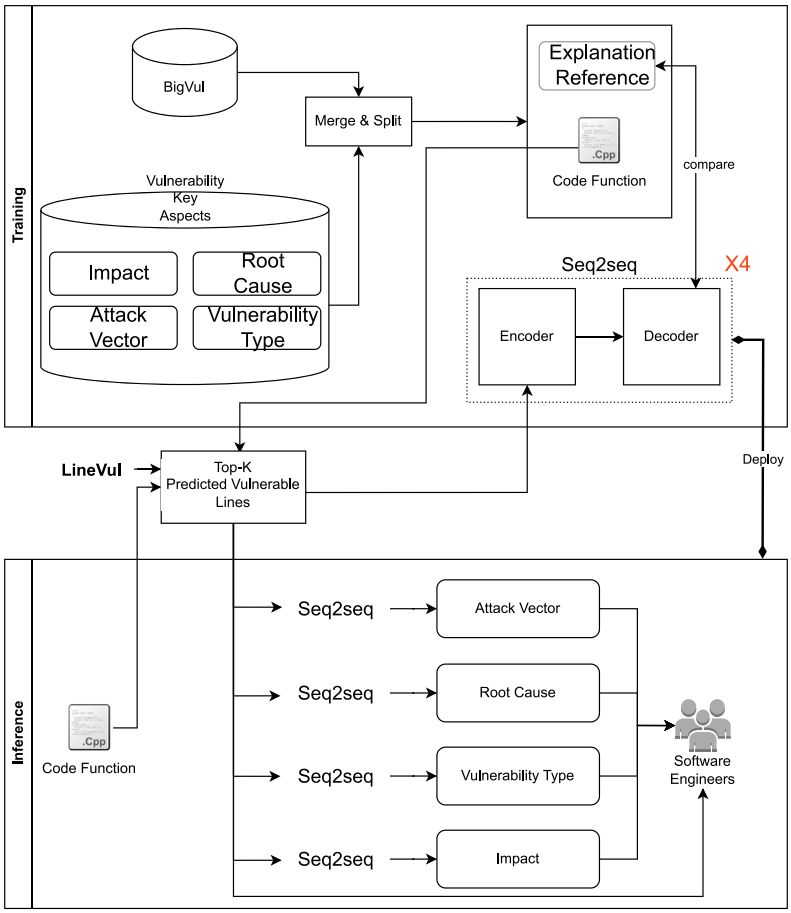

Fig. 1. Our explanation framework aims to provide software practitioners with four key aspects for a given k-vulnerable code lines inside a predicted function. It consists of four distinct Seq2seq models, all trained with real security analysis reports from the Vulnerability Key Aspects (Sun et al., 2023) dataset and its associated vulnerable code function in BigVul (Fan et al., 2020). Red arrows indicate data flow during the inference phase, while black arrows represent the training phase. Small computer icons represent deployed models after training.

Table 1
Details of data after merging BigVul and Vulnerability Key Aspects.

	aspect length (Mean, STD, Max)	#train	#val	#test
Impact	56, 24.6, 167	7,032	782	1,954
Root Cause	52, 26.1, 153	3,431	382	954
Attack Vector	45, 21.5, 146	4,858	540	1,350
Vulnerability Type	19, 4.8, 53	3,870	431	1,076

1. In the first step, we filter the code functions labeled as "vulnerable" in BigVul. This step emulates a real-world context where users will employ a vulnerability prediction model and then use an explanation method on vulnerable functions. This step is consistent with methodologies proposed in the literature (Li et al., 2021; Nguyen et al., 2025).
2. We obtain the positive vulnerability function, which is obtained from step #1. In parallel, we extract the key aspects for each CVE in VKA. Since both BigVul and VKA datasets use CVE as the primary key, we merge the two datasets based on CVEs. Finally, a record is a pair of vulnerable code functions and its explanation text.
3. Finally, we split the datasets into training, validation, and test subsets. Following the common practice, we allocate 70% of the data for training, 10% for validation, and 20% for testing.

The details and statistics of these datasets are presented in Table 1. The sizes of those datasets range from 4767 records (the Root Cause) to 9767 records (Impact aspect). The Vulnerability Type dataset has the fewest tokens, with a maximum length of 53 tokens, whereas the one for the Impact has the most tokens, with a maximum length of 167 tokens.

3.3. Key aspect generations

After dividing our data into four distinct datasets, each dedicated to a specific key aspect of vulnerability, i.e., Attack Vector, Root Cause, Impact, and Vulnerability Type, we train four independent models to explain each key aspect. Each model is designed to generate explanations for a specific key aspect based on the given code functions. Our models employ a sequence-to-sequence (seq2seq) transformer architecture (Vaswani et al., 2017). Essentially, each model processes a sequence of input tokens and generates a corresponding sequence of output tokens.

The transformer model consists of two main components: the encoder and the decoder. We illustrate the seq2seq transformer architecture in Fig. 3. The input, a sequence of words, is processed by a tokenizer, where it is broken down into individual tokens and then transformed into vector representations. That representation is known as embeddings, which are fed into the encoder component. The encoder will convert it into a sequence of hidden states that encapsulate the contextual information of the input. The decoder is involved after the encoder generates output autoregressively, conditioned on the previous tokens, starting with a special token. In Fig. 3, we assume that the decoder will generate three tokens; hence, it will repeat the process three times. We use "< s >" to represent the start token, and the decoder

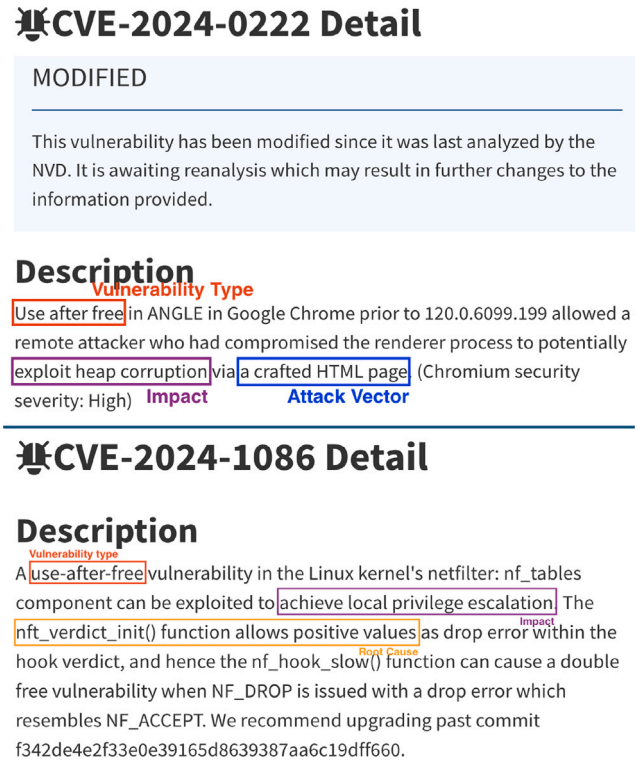

Fig. 2. Screenshots from CVE-2024-0222 and CVE-2024-1086 reports. We also annotate key aspects for each report. CVE-2024-0222 has Vulnerability Type, Impact, and Attack Vector, while CVE-2024-1086 does not have Attack Vector, but Root Cause.

produces the first output token, "Out". Next, "Out" is fed for the next iteration to produce "of". Finally, the decoder produces the last token, "bounces", from "of". For each iteration, the decoder always takes encoder's output, that is called the cross-attention mechanism (Vaswani et al., 2017). After the second iteration, the decoder will take the output token from the previous iteration as the input for the next iteration.

Transfer learning, a technique widely applicable to various deep learning architectures including transformers, recurrent neural networks, and convolutional neural networks (Goodfellow et al., 2016, p.526), offers the advantage of accelerating model convergence and reducing computational costs. This technique involves leveraging a pre-trained model as a starting point for a new task. In our study, we focus on applying transfer learning by leveraging a pre-trained model in either the encoder-only or both encoder and decoder. The various adaptations of this model for generating explanations for key aspects are detailed in the following subsections.

3.3.1. Pre-trained encoder

Using a pre-trained encoder is one of two strategies we apply in our study. This strategy is to reuse the pre-trained encoder, trained on other tasks or different datasets. However, such shared models do not have a decoder component, which let downstream tasks decide how to use them. Thus, to utilize this encoder pre-trained, a decoder must stack on top of the encoder, forming a complete model, and then fine-tune the new model on our datasets.

A pre-trained encoder, specific to the programming language, is CodeBERT (Feng et al., 2020). CodeBert was trained on 2.1 million bimodal data points (pairs of natural language and code) and 6.4 million unimodal code samples (code without paired natural language and vice versa) from six programming languages: Go, Java, JavaScript, PHP, Python, and Ruby. The training utilized two objectives: Masked Language Modeling (MLM) and Replaced Token Detection (RTD). Given that the BigVul dataset comprises C/C++ data, and CodeBERT does not train on C/C++, we opted for a different version of CodeBert, referred to as **CodeBert-C**++ from Zhou et al. (2023). Through our study, we name CodeBERT in Feng et al. (2020) as **CodeBert-base**. CodeBert-C++ was further trained on 87.73 GB of data from the Github Code Clean dataset (Anon, 2023).

As we need to develop a decoder component, we followed Vaswani et al. (2017). This component comprises a sequence of transformer decoder blocks, followed by a dense layer and a tanh activation function, which introduces non-linearity and ensures that the output values fall within a specified range. Inspired by Vaswani et al. (2017), we design two versions of our decoder component. The first version consists of a stack of six transformer decoder blocks, while the second features twelve layers of transformer decoders. When integrated with a type-specific encoder component, our decoder forms a complete model, with the number of layers reflected in the model's name. In the end, we construct three distinct models: **CodeBert-base + 6 decoder layers**, **CodeBert-C++ + 6 decoder layers**, and **CodeBert-C++ + 12 decoder layers**.

3.3.2. Pre-trained encoder–decoder

Unlike the pre-trained encoder, the pre-trained encoder–decoder strategy involves reusing whole models (both the encoder and decoder) to fine-tune our datasets. In this study, we employ a pre-trained model, namely CodeT5+ (Wang et al., 2023c). Unlike CodeBERT, which was trained on 8 million files, CodeT5+ leverages a significantly larger dataset, comprising over 37 million files. Additionally, CodeT5+ employs a code-customized Byte-level BPE tokenizer, as proposed by Radford et al. (2019), in contrast to CodeBERT's use of a standard Byte-Pair Encoding tokenizer (Sennrich et al., 2016). Furthermore, CodeT5+ underwent training on code data involving diverse tasks, including span denoising, causal language modeling, text-code contrastive learning, and text-code matching.

Although several versions of CodeT5+ are available, with pre-trained parameters ranging from 220 million (**CodeT5+-220M**) to 16 billion, we opted for the CodeT5+-220M version due to our computational resource constraints. Moreover, our study does not explore prompt-based methods in LLMs, and the selected versions do not require additional prompts for the decoder component.

3.4. Experimental setup

All our experiments were run on two NVIDIA GeForce RTX 3090 (24 GB) graphic cards. For all experiments, we used the same batch size, optimizer, learning rate, and maximum epoch, which are 40, AdamW, 5e−5, 100 respectively. We report the number of hyper-parameters in Table 2.

In the training phase, only training and validation datasets were used. During training, the model parameters were adjusted to minimize the difference between their generated results and expected results through the cross entropy loss function.[11] The validation datasets were used to determine the optimal set of weights that achieve the highest ROUGE across all epochs. However, due to our computational constraint, when we trained the CodeT5+-220M, we set the floating point precision (Micikevicius et al., 2018) to 16-bit.

Following this, in the evaluation, we assess the performance of the models using the test dataset, applying the models that have been fine-tuned with the previously identified optimal weights. During the inference phase, we implemented the beam search technique (Tillmann and Ney, 2003), widely used in natural language processing tasks. Unlike greedy decoding, which selects the next token based solely on the highest immediate probability, beam search finds a set of the most probable tokens for each step. Those tokens are then used to find the

[11] https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

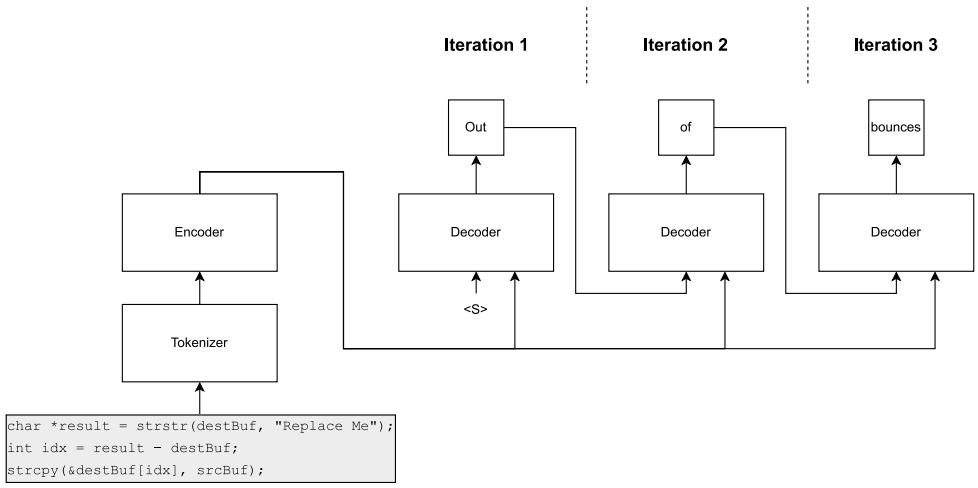

Fig. 3. Seq2seq Transformer architecture (Vaswani et al., 2017).

Table 2
The number of hyper-parameters.

	Model size
CodeBert-base + 6 decoder layers	172 million
CodeBert-C++ + 6 decoder layers	
CodeBert-C++ + 12 decoder layers	220 million
CodeT5+-220M	220 million

most probable next tokens for each. That process repeats to expand the set of possible sequences, and the final sequence is selected based on the cumulative probability score. Thus, beam search effectively explores a wider range of possible predictions, increasing the chances of finding a more accurate output. We employed a beam size of 5 for all experiments. At this phase, we use both ROUGE, BLEU, and BERTScore to evaluate the performance of our proposed framework and other baselines.

4. Evaluation

We investigate which setting of our framework can produce superior performance in generating explainable key aspects of vulnerabilities. Besides, we plan to compare it with other state-of-the-art approaches. Thus, we break down our goals by addressing the following research questions (RQ):

- ***RQ1:*** Which of the pre-trained models for source code produce superior performance in generating explainable key aspects of vulnerabilities? **Motivation:** Since we leverage pre-trained models to develop our framework, we need to determine which pre-trained model is the most effective in generating key aspects of vulnerabilities.
- ***RQ2:*** How is the performance of our proposed framework compared to the state-of-the-art methods for key aspect generation? **Motivation:** In this research question, we plan to compare our framework with other methods that can be used to generate key explanations. The first method is from Sun et al. (2023), which, to the best of our knowledge, is the state-of-the-art method in generating key aspects explanation for vulnerability prediction. Other methods are Retrieval Augmented Generation (RAG) (Lewis et al., 2020a) and Few-shot Learning (Brown et al., 2020), which are state-of-the-art in LLM-based text generation.
- ***RQ3:*** How does the number of lines of code selected for explanation influence the performance of the generated key aspects? **Motivation:** In RQ1 and RQ2, we employ k-top lines of code ($k = 10$) that are likely vulnerable to obtaining the prediction's output. In this research question, we investigate how the k factor influences the performance of explanation outcomes.

4.1. Baseline approaches

We present the baseline approaches in details, that we compare with our framework in RQ2.

BART-CodeBERT: Sun et al. (2023) propose a Seq2seq model that is a mixture of BART (Lewis et al., 2020b) and CodeBERT to generate explanations. Thus, we name their model as **BART-CodeBERT**. In addition, we note that our study and Sun et al. (2023) both apply transformer, but the objectives are different. Sun et al. provide key aspects in explaining merge requests, which could be silent vulnerability patches, enabling software developers to quickly apply fixes as soon as the requests are merged. However, their data inputs differ in which their Seq2seq takes a pair of text descriptions on merge requests and code change fragments provided by the git diff. The text description goes to the BART-encoder, and the git diffs are entered into CodeBERT, later, both input are merged at the cross-attention layer. To make Sun et al.'s model fit our study, we used their configurations (including their pre-training models such as BART, and CodeBERT) and then re-train their approach with slight modification at the input, which we feed the full version of the code function into the BART-encoder while its top 10 lines, generated by LineVul, into CodeBERT. We illustrate our modification on the BART-CodeBERT model's input to our dataset in Fig. 4.

Retrieval Augmented Generation (RAG): An alternative approach is to use RAG (Lewis et al., 2020a), which is a combination of language models and an indexing database for retrieving information. That strategy brings the benefit from a trained language model, which is not needed for fine-tuning while reducing the hallucination phenomenon from that model.

RAG involves a database that supports a vector datatype named a Vector Database (see Fig. 5). In addition, RAG needs a trained model capable of transforming input into a vector. That vector is called embedding, as the vector holds abstract information from the trained model, is represented in a real vector. Before making a query, all train records go through the models, then themselves and their corresponding key explanations (herein metainfo) are inserted in the database. That means we do not need to fine-tune the model, but we need to transform the input into a vector. During running time, each second in test set is transformed into a vector, and then the database searches the most similar LoC code function's vector based on vector similarity. Once found, metainfo from the record is returned.

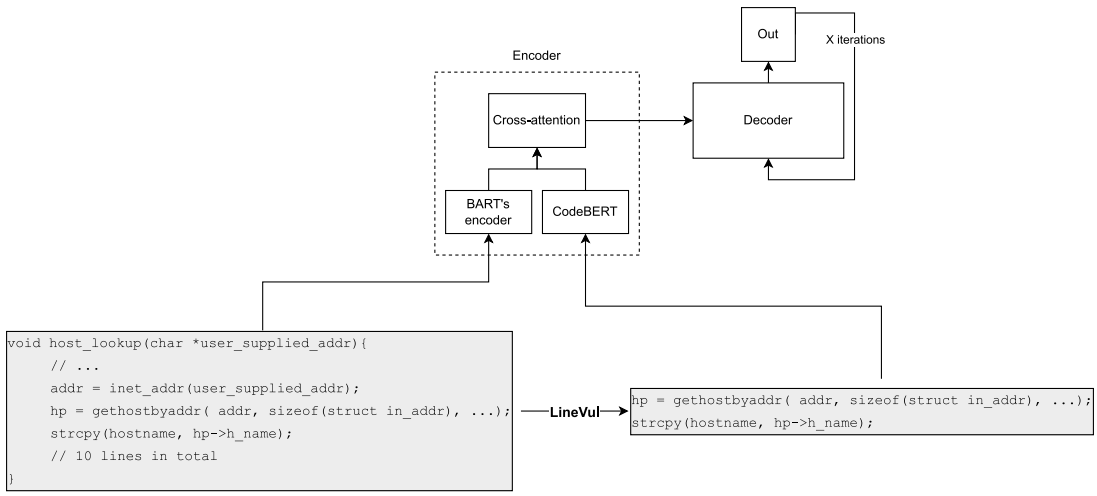

Fig. 4. Sun et al. (2023) mixed CodeBERT-base with BART's encoder. We reuse their model by feeding full code function into BART's encoder and top k lines into CodeBERT-base.

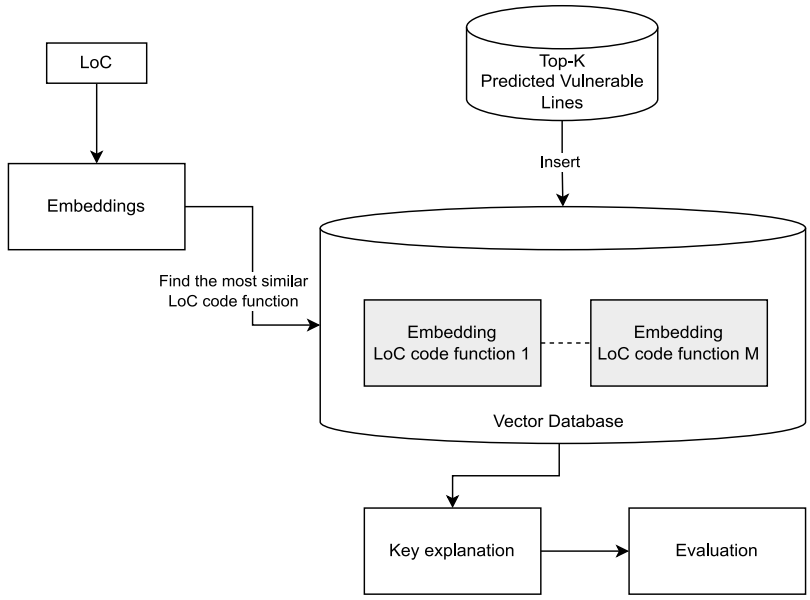

Fig. 5. We illustrate how we apply RAG for a key aspect dataset. RAG needs a vector database and a pre-trained model to transform input into a vector. For each dataset, a separate vector database is created. The vector database searches the most similar records based on vector similarity. Since a vector is tagged with key aspects, the most similar record's key aspect is returned.

Recall in the RQ1 (Section 4.3), CodeT5+-220M is the most robust pre-train for programming, in this research question, we use it as a trained model. As CodeT5+ is a transform model, technically, we only use its encoding part and aggregate its sequence of hidden states inside the encoder into a vector. However, we used a version embedding model of CodeT5+-220M, which is provided by CodeT5+ authors (Wang et al., 2023c). Along side with CodeT5+-200M, we use the embedding model - *SRoberta*,[12] which is used in a RAG-based code retrieving study (Nashid et al., 2023). We use abbreviations to refer to them, respectively, RAG (CodeT5+) and RAG (BM25).

Regarding the vector database, we deployed ElasticSearch.[13] With ElasticSearch's support, we used the dot product operator to compute vector similarity and kNN to retrieve the most similar record. We provide scripts and Dockerfile to reproduce our study's repository.

Our populating data procedure is following: for each dataset (Impact, Root Cause, Attack Vector, and Vulnerability Type), we loop over all train records, feeding the top k lines to the embedding model, and then create a pair of embedding vectors and key aspect. Those pairs are later inserted into the database. That leads each dataset will have a separate vector database. To evaluate RAG, we repeat the same procedure on the test dataset, but instead of inserting it, we query the database to find the most similar record for each test candidate.

Few-Shot Learning: With RAG-based approaches, a key explanation is retrieved without a learning process. The few-shot learning (Brown et al., 2020) is slightly different, where it leverages prompt engineering to interact with an LLM after embedding data, aiming to generate output on very small examples. In relation to software engineering, Nashid et al. (2023) apply few-shot learning to generate code fixes given an incomplete code snippet. We utilized Nashid et al.'s

[12] https://huggingface.co/flax-sentence-embeddings/st-codesearch-distilroberta-base

[13] https://www.elastic.co/elasticsearch

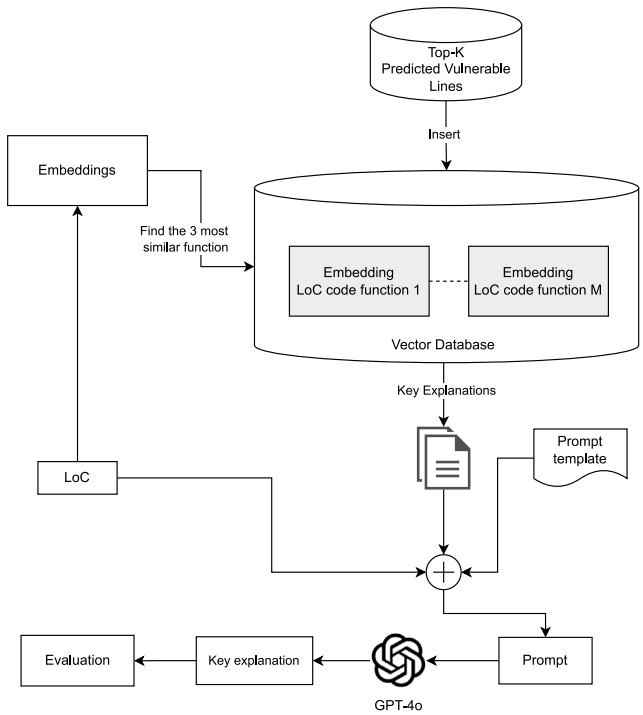

Fig. 6. We illustrate how we apply few-shot learning in our study.

framework to generate key aspects for vulnerability prediction. The approach's overview is illustrated in Fig. 6 and described as follows.

Similar to RAG (BM25), each records in the train set is fed to SRoberta—an embedding model, which is also used in Nashid et al.'s original work, then their embedding vector and its metainfo are stored in a database. During the test time, given a test inquiry, we transform it into a vector and then query the database. Different from RAG, in few-shot learning, we aim to find three records that are most similar instead of one. From those three records' metainfo, and a prompt template, we then generate a prompt which is used to interact with a LLM model to obtain key aspects.

Nashid et al.'s prompt template has 3 parts: *System*, *Demonstrations*, and *User Input*. We slightly modified the System to fit our purpose and kept the rest of the template. Our system part is *You are a software engineer in C/C++ and know about vulnerability and cyber-security. Given a code snippet, you will return a line of text, which is* ***KEY, KEYDEF***. Our prompt introduces KEY and KEYDEF, aiming to provide to an LLM a definition of a key aspect that we want it to generate. Therefore, we have four distinct prompts for four key aspects. In the demonstration part, as in Nashid et al. we create pairs of input (User) and output (AI), which is the LLM will take input from users and what LLM should generate. However, we use the three most similar LoCs and their metainfo as three demonstrations. Our changes results each demonstration is an example to LLM, when users give a LoC, what the LLM should generate. Finally, the user input part is a holder where we put a LoC that we want to generate key aspects. We depict the prompt template in Fig. 7.

We also use the same pre-trained model that Nashid et al.'s work converts a LoC into a vector. Besides, we use ElasticSearch as our main vector databases, and we use the dot product operator to compute vector similarity and kNN to retrieve three most similar LoC. The LLM that we used in this experiment is GPT-4o, a LLM model from OpenAI, backs Github Copilot for AI-base pair programming. We provide scripts and Dockerfile to reproduce our implementation at our study's repository.

4.2. Evaluation metrics and hypothesis testing

To measure performance, we employ evaluation metrics: ROUGE (Lin, 2004), BLEU (Papineni et al., 2002), and BERTScore (Zhang et al., 2020). We first describe them and then explain how we conduct hypothesis testing.

4.2.1. Evaluation metrics

We employ the ROUGE score (herein ROUGE) to estimate the effectiveness of our model (Lin, 2004). Specifically, ROUGE computes the similarity between the generated text and the reference explanation (human-annotated text) regarding the number of words in the reference present in the generated text. In addition to the ROUGE, we use the BLEU score (herein BLEU) for our evaluation purpose (Papineni et al., 2002). Different from ROUGE, BLEU measures how many words in the generated text appear in reference text.

ROUGE has various versions, i.e., ROUGE-1, ROUGE-2, and ROUGE-L, which distinguish themselves based on the size of the n-grams they consider. ROUGE-1 finds a single word, while ROUGE-2 seeks two consecutive words. Unlike those versions, ROUGE-L identifies the longest sub-sequence between generated and reference sequences. Similar to ROUGE, BLEU also has different versions: BLEU-1, BLEU-2, BLEU-3, and BLEU-4, which represent the number of n-grams considered. BLEU-1, BLEU-2, BLEU-3, and BLEU-4 focus on unigrams, bigrams, trigrams, and four-grams, respectively. We utilize all versions for a comprehensive comparison. For brevity in our evaluation section, we refer to ROUGE-1, ROUGE-2, and ROUGE-L as R-1, R-2, and R-L, respectively. In parallel, we refer to BLEU-1, BLEU-2, BLEU-3, and BLEU-4 as B-1, B-2, B-3, and B-4, respectively.

In order to offer deeper insights into the effectiveness of our proposed framework, BERTScore (Zhang et al., 2020) is also considered as an evaluation metric. Different from ROUGE and BLEU, BERTScore calculates the similarity between the generated text and the reference text based on the cosine similarity between embedding vectors of the generated and reference text. Since BERTScore uses BERT (Devlin et al., 2019), BERTScore considers contextual meaning similarity at the sentence level; thus, similarity is highest when two candidates have close meaning. In particular, BERTScore computes recall, precision, and F1-score to evaluate the generated text and the reference text. Throughout our study, we will report F1-score as it balances recall and precision.

4.2.2. Hypothesis testing

To compare the effectiveness of two arbitrary methods, we perform hypothesis testing to confirm the statistical significance between them. We label the two methods in question as A and B and establish the hypothesis test to check whether A is better than B.

$$\begin{aligned} H0 &: \mu_A - \mu_B = 0 \\ H1 &: \mu_A - \mu_B > 0 \end{aligned} \tag{1}$$

where μ_A and μ_B are the mean of results from two compared methods, respectively. We reject $H0$ if the p-value is less than the significant level, which we set at 0.05. Before conducting the comparison test, both A and B must go through a normality test to check whether they are normally distributed. We apply Shapiro and Wilk (1965) for this purpose. If Shapiro–Wilk significantly indicates both A and B follow the normal distribution, we will perform the paired t-test (Yuen and Dixon, 1973). Otherwise, the Wilcoxon Signed-rank test (Muller, 1989), which is a non-parametric test, is conducted. In case the normality test prefers t-test, we then apply Cohen's D (Muller, 1989) to measure the effect size. On the other hand, we report the effect size using Vargha and Delaney (2000).

4.3. Model quality and choice of pre-trained model (RQ1)

Since a code function will go through LineVul to obtain k-line of code that is likely vulnerable, we first select $k = 10$, which is also used as the default parameter in LineVul. Besides, we found that all yielded results are normal distribution, which leads t-test, and Cohen's D are used in this section.

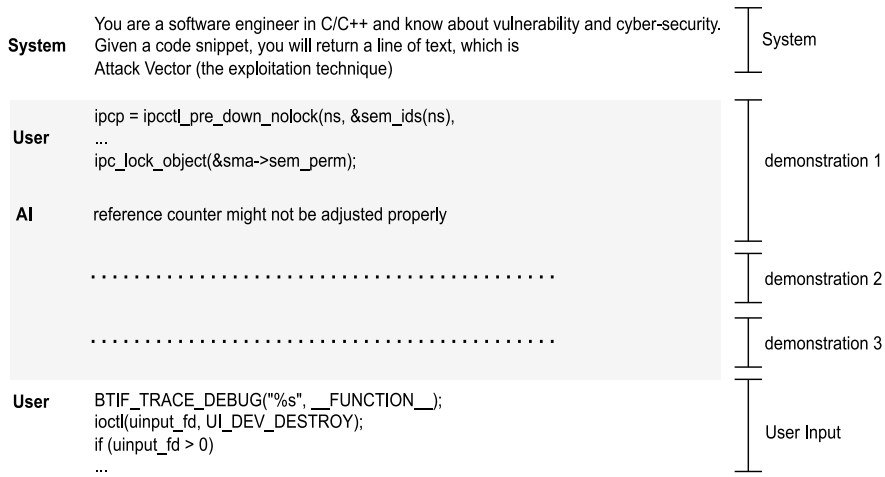

Fig. 7. We illustrate our prompt template for attack vector. There are three parts: System, Demonstrations, and User Input. Demonstrations will be populated with the three most similar LoCs and their key aspects found in the vector database.

Table 3 offers a detailed comparative analysis of two distinct pre-training strategies: one employing an encoder-only approach with three variants of CodeBERT, and the other utilizing an encoder–decoder configuration with two variants of CodeT5+. Across all evaluated key aspects–*impact*, *root cause*, *attack vector*, and *vulnerability type*–the models initialized with CodeT5+ consistently outperform variances based on CodeBERT. For example, in ROUGE-L under *Attack Vector*, CodeT5+-220 m achieves 17% higher than CodeBERT-base, 12.8% higher than CodeBERT-C++ (6 decoder layers), and 15.4% higher than CodeBERT-C++ (12 decoder layers). The trend is maintained in the assessment of *impact* and *vulnerability type*, with CodeT5+-220 m achieving higher ROUGE scores, indicating a clear advantage over the encoder-only strategy. However, we observe CodeT5+-220 m creates a small difference in ROUGE-2 under *vulnerability type* aspect, for example, 0.641 (CodeT5+-220 m) versus 0.625 (CodeBERT-C++ (12 decoder layers)).

We employ the hypothesis testing framework outlined in Section 4.2.2 to statistically compare the performance of CodeT5+ pre-trained models and other models. Specifically, we define two sets of models: Set A, containing CodeT5+-220M, and Set B, comprising the other evaluated models. We construct eight unique model pairs and apply three distinct metrics to each, with results presented in Table 6. For clarity, the models from Set A and Set B are displayed as columns and rows, respectively. Each cell in the table specifies the p-values derived from the statistical tests. ROUGE scores are all normally distributed, thus we apply the paired t-test to compare the performance of two models. We find that models pre-trained with CodeT5+ demonstrated a statistically significant improvement over CodeBERT-based models, with effect sizes varying from 0.11 to 0.88. It is, however, noteworthy that in certain model comparisons, the effect sizes for ROUGE-2 were smaller than those for ROUGE-1 and ROUGE-L. For example, within the *vulnerability type* aspect revealed a smaller effect size for ROUGE-2. In addition, from the test, we observe there is no significant difference in ROUGE-2 between CodeT5+-220M versus CodeBERT-C++ (6 decoder layers) and CodeBERT-C++ (12 decoder layers), which confirms the observation in Table 3.

We also report BLEU scores in Table 4. CodeT5+-200M outperforms others in all key aspects. Specifically, CodeT5+-200M's BLEU score is higher, approximately 30% in Impact, 28% in Root Cause, and 14% in Attack Vector. Regarding *vulnerability type*, we observe that CodeT5+-200M's performance is slightly higher only in BLEU-4. BLEU-1, BLEU-2, and BLEU-3 are relatedly modest. For instance, BLEU-1 compared to CodeBERT-base is less than 0.157, and BLEU-3 is less than 0.05. In fact, in Table 7, the hypothesis can confirm that only *vulnerability type* CodeT5+-200M cannot significantly outperform others in BLEU-2, BLEU-3, BLEU-4.

As we mentioned in Sub- Section 4.2.1, we next report the performance where context similarity is considered, which is BERTScore. In Table 5, we observe that CodeT5+-200M outperforms others in all key aspects. Specifically, at Table 8, under *impact*, and *vulnerability type*, effect sizes vary from 0.560 to 0.881, which means the difference is large. In the two remaining aspects (*root cause*, and *attack vector*), the effect sizes vary from 0.172 to 0.466, which means the difference ranges from small to medium.

> Pre-trained using CodeT5+ is the pre-trained model that performs best in generating key aspects of vulnerabilities. It statistically outperforms ROUGE and BLEU scores under *impact*, *root cause*, and *attack vector*. In BertScore, CodeT5+ also statistically outperforms others across all key aspects.

4.3.1. Output illustration for the motivating example

As Listing 1, lines from #8, 9, 10 are predicted, they are fed into our four CodeT5+-220M explanation models, then give developers output: Vulnerability Type is *memory leak*, Attack Vector is *via a crafted application*, Impact is *execute arbitrary code on the system*, and Root Cause is *does not ensure that clear calls interact properly with the state of a draw buffer*. Our generated explanations convey sufficient information which helps understand the predicted vulnerability. Specifically, use-after-free vulnerability is similar to memory leak vulnerability[14] since they both are an unintentional form of memory consumption. Also, in the issue tracker, we know that experts use a snippet of JavaScript code inside an HTML page to exploit the vulnerability, which is our explanation for the Attack Vector. In addition, the experts mentioned that they find that attackers have GPU execute commands,[3] which is similar to our explanation for the Impact (see Table 8).

4.4. Baseline comparison (RQ2)

We report results in Table 9 (ROUGE), Table 11 (BLEU), and Table 13 (BertScore). Similar to RQ1, we also perform hypothesis testing, which is mentioned in Section 4.2.2. In this section, we found *vulnerability type* aspect's BLEU scores are not normally distributed, thus we apply the Wilcoxon Signed-rank test to compare the performance of two models. For those that are not normally distributed, we mark them within our hypothesis result tables (10, 12, 14). For testing purpose,

[14] https://owasp.org/www-community/vulnerabilities/Using_freed_memory

Table 3
A comparison between pre-training strategies in ROUGE metrics. R1, R2, RL stand for ROUGE-1, ROUGE-2, ROUGE-L respectively.

	Impact			Root cause			Attack vector			Vulnerability type		
	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L
CodeBert-base + 6 decoder layers	0.626	0.570	0.620	0.594	0.567	0.592	0.723	0.660	0.721	0.608	0.587	0.608
CodeBert-C++ + 6 decoder layers	0.701	0.658	0.696	0.629	0.604	0.627	0.753	0.695	0.753	0.645	0.627	0.645
CodeBert-C++ + 12 decoder layer	0.693	0.654	0.688	0.627	0.600	0.626	0.738	0.680	0.736	0.647	0.625	0.645
CodeT5+-220M	**0.896**	**0.847**	**0.896**	**0.815**	**0.748**	**0.813**	**0.851**	**0.774**	**0.850**	**0.776**	**0.641**	**0.775**

Table 4
A comparison between pre-training strategies in BLEU metrics. B1, B2, B3, B4 stand for BLEU-1, BLEU-2, BLEU-3, BLEU-4 respectively.

	Impact				Root cause				Attack vector				Vulnerability type			
	B-1	B-2	B-3	B-4	B-1	B-2	B-3	B-4	B-1	B-2	B-3	B-4	B-1	B-2	B-3	B-4
CodeBert-base + 6 decoder layers	0.638	0.581	0.561	0.542	0.584	0.554	0.542	0.534	0.714	0.647	0.633	0.626	0.611	0.596	0.634	0.666
CodeBert-C++ + 6 decoder layers	0.725	0.684	0.671	0.660	0.623	0.594	0.583	0.576	0.714	0.647	0.633	0.626	0.646	0.634	0.659	0.698
CodeBert-C++ + 12 decoder layer	0.719	0.655	0.640	0.630	0.610	0.578	0.565	0.555	0.719	0.655	0.640	0.630	0.648	0.633	0.657	0.697
CodeT5+-220M	**0.916**	**0.876**	**0.842**	**0.812**	**0.820**	**0.759**	**0.723**	**0.705**	**0.852**	**0.782**	**0.747**	**0.722**	**0.768**	**0.653**	**0.693**	**0.852**

Table 5
A comparison between pre-training strategies in F1-score of BERTScore.

	Impact	Root cause	Attack vector	Vulnerability type
CodeBert-base + 6 decoder layers	0.935	0.938	0.953	0.941
CodeBert-C++ + 6 decoder layers	0.948	0.944	0.953	0.946
CodeBert-C++ + 12 decoder layer	0.947	0.943	0.956	0.947
CodeT5+-220M	**0.981**	**0.967**	**0.965**	**0.979**

Table 6
We report effect size in hypothesis testing as we mentioned in Eq. (1), where methods in columns (A) against methods in rows (B). R-1, R-2, R-L stand for ROUGE-1, ROUGE-2, ROUGE-L, respectively. Note that we do not need to compare CodeT5+-220M with itself. If we are not able to reject the null hypothesis, we mark it as X.

	CodeT5+-220M			
	R-1	R-2	R-L	
CodeBert-base + 6 decoder layers	0.875	0.787	0.884	Impact
CodeBert-C++ + 6 decoder layers	0.636	0.537	0.645	
CodeBert-C++ + 12 decoder layers	0.653	0.550	0.661	
CodeBert-base + 6 decoder layers	0.579	0.421	0.573	Root Cause
CodeBert-C++ + 6 decoder layers	0.492	0.338	0.488	
CodeBert-C++ + 12 decoder layers	0.498	0.347	0.494	
CodeBert-base + 6 decoder layers	0.395	0.281	0.394	Attack Vector
CodeBert-C++ + 6 decoder layers	0.395	0.281	0.394	
CodeBert-C++ + 12 decoder layers	0.352	0.234	0.350	
CodeBert-base + 6 decoder layers	0.425	0.116	0.425	Vulnerability Type
CodeBert-C++ + 6 decoder layers	0.335	X	0.335	
CodeBert-C++ + 12 decoder layers	0.332	X	0.332	

we call the CodeT5+-200M pre-trained model as A, and other baseline approaches as B.

From Table 9, our framework outperforms other baselines in ROUGE scores. Later, our hypothesis test in Table 10 confirms that result. In general, BART-CodeBERT is the second-best baseline, following few-shot learning, while RAG-based baselines give poor performance. For instance, the effect size when CodeT5+-200M against BART-CodeBERT under ROUGE-2 under *impact* is 0.707 while collating against RAG (BM25), the effect size is 6.282.

In terms of BLEU scores, we report in Table 11, and the associated hypothesis test in Table 12. In general, CodeT5+-200M maintains the best performance, however, there is some noise in the results. Firstly, CodeT5+-200M's performance is slightly better than BART-CodeBERT in *vulnerability type* aspect. Specifically, in BLEU-1, the effect size is 0.334, then dropping in BLEU-2, and BLEU-4 to 0.015, and 0.033, respectively. CodeT5+-200M is unable to reject the null hypothesis in BLEU-2. Secondly, under *root cause* and *attack vector* aspect, we find that, only occurring in CodeT5+-200M versus the few-shot learning, the size effects are significantly larger than other pairs. To demonstrate, under *root cause*, the RAG (BM25)'s effect size is 0.467 (BLEU-3); simultaneously, the effect size of the few-shot learning is 2.007. Thirdly, under *vulnerability type*, from our validating the normality in BLEU-4, RAG (CodeT5+) and RAG (BM25) results are not normally distributed. Thus, we use the Wilcoxon Signed-rank test to compare the performance of two models and use Vargha and Delaney to measure the effect size. We observe that CodeT5+-200M still outperforms them, but the effect size is small, 0.08 against RAG (CodeT5+) and 0.079 against RAG (BM25).

Next, we report the BertScore in Table 13, and the hypothesis test in Table 14. All BertScore results are normally distributed; thus, we apply the paired t-test to compare the performance of the two models. The pattern we observe in BertScore is similar to the one in ROUGE and BLEU.

Among baselines, RAG-related baselines and the few-shot Learning do not perform well in our problem, specifically in impact and vulnerability type aspects. For instance, in terms of BERTScore, under impact and vulnerability type, those group's achieved F1-score is less than 0.872. In parallel, BART-CodeBERT and CodeT5+-200M, the F1-score is more than 0.943. The few-shot learning is somewhat better than RAG-related baselines, but from our experiment, it is heavily hallucinate in generating key aspects. That pattern is observed through ROUGE and BLEU scores, where the few-shot learning's performance is lower than RAG-related baselines approximately 20% to 30%. On the other hand, both BART-CodeBERT and CodeT5+-200M involves in fine-tuning activity, which results in a better performance and less hallucination.

Table 7
We report p-value and effect size in hypothesis testing as we mentioned in Eq. (1) where methods in columns (A) against methods in rows (B). B1, B2, B3, B4 stand for BLEU-1, BLEU-2, BLEU-3, BLEU-4 respectively. Note that we do not need to compare CodeT5+-220M with itself. If we are not able to reject the null hypothesis, we mark it as X.

	CodeT5+-220M				
	B-1	B-2	B-3	B-4	
CodeBert-base + 6 decoder layers	0.821	0.765	0.641	0.490	Impact
CodeBert-C++ + 6 decoder layers	0.597	0.526	0.398	0.276	
CodeBert-C++ + 12 decoder layers	0.613	0.536	0.408	0.291	
CodeBert-base + 6 decoder layers	0.563	0.412	0.280	0.203	Root Cause
CodeBert-C++ + 6 decoder layers	0.476	0.328	0.205	0.147	
CodeBert-C++ + 12 decoder layers	0.487	0.339	0.212	0.152	
CodeBert-base + 6 decoder layers	0.405	0.319	0.241	0.167	Attack Vector
CodeBert-C++ + 6 decoder layers	0.405	0.319	0.241	0.167	
CodeBert-C++ + 12 decoder layers	0.367	0.273	0.192	0.122	
CodeBert-base + 6 decoder layers	0.370	X	X	X	Vulnerability Type
CodeBert-C++ + 6 decoder layers	0.284	X	X	X	
CodeBert-C++ + 12 decoder layers	0.280	X	X	X	

Table 8
We report p-value and effect size in hypothesis testing as we mentioned in Eq. (1) where methods in columns (A) against methods in rows (B). Note that we do not need to compare CodeT5+-220M with itself, and because there is no case that we cannot reject $H0$, X symbol is not used in this table.

	CodeT5+-220M	
CodeBert-base + 6 decoder layers	0.881	Impact
CodeBert-C++ + 6 decoder layers	0.634	
CodeBert-C++ + 12 decoder layers	0.646	
CodeBert-base + 6 decoder layers	0.466	Root Cause
CodeBert-C++ + 6 decoder layers	0.367	
CodeBert-C++ + 12 decoder layers	0.389	
CodeBert-base + 6 decoder layers	0.223	Attack Vector
CodeBert-C++ + 6 decoder layers	0.223	
CodeBert-C++ + 12 decoder layers	0.172	
CodeBert-base + 6 decoder layers	0.651	Vulnerability Type
CodeBert-C++ + 6 decoder layers	0.659	
CodeBert-C++ + 12 decoder layers	0.560	

Our framework significantly outperforms baselines in ROUGE, BLEU, and BERTScore. The only exception occurs in BLEU-3 under *vulnerability type* aspect, where CodeT5+-200M does not significantly outperform BART-CodeBERT. Also, from our experiment, we observe that fine-tuning pre-trained models can achieve better performance than RAG-related baselines and the few-shot learning.

4.5. How percentage of top lines influence performance (RQ3)

In this question, the percentage of top lines calculates the number of lines. In fact, we vary the percentage LoC (%LoC) from 10%, 20%, …, 90%, 100% (full), then performing evaluating. It is worth noting that we establish a condition that the number of lines of code corresponding to the percentage must be at least 1 in the input code function to ensure that lines from code functions with few lines are not empty after applying percentage line extraction.

We present this RQ's results in Fig. 8, where four sub-figures represent key aspects. The X-axis is the percentage of top lines and the Y-axis is the ROUGE scores. In general, all ROUGE metrics exhibit the same pattern. In the aspects of Impact, Root Cause, and Attack Vector, the ROUGE score gradually increases as the %LoC increases, despite minor fluctuations. This trend peaks between 40% and 85%, after which it decreases. Notably, Impact does not experience a significant drop after reaching its peak. In contrast, Vulnerability Type shows a different pattern, where the ROUGE score continues to increase even when %LoC exceeds 90%, despite a slight dip from 70% to 85% before increasing again.

The above observations suggest a trade-off between the percentage of top lines and the ROUGE scores. Using a low %LoC results in low scores, while using a high %LoC does not necessarily lead to better performance. The optimal %LoC is 80% for Impact, 60% for Root Cause, and 70% for Attack Vector. For Vulnerability Type, the best ROUGE scores are achieved with 100% of LoC, but 70% is considered an optimal choice as the difference in scores is less than 0.01. Despite this, a high %LoC can be inconvenient for security analysts (Parnin and Orso, 2011). Thus, a common choice is $k = 10$.

Next, we aim to determine whether $k = 10$ is a suitable choice. In Fig. 8, we draw vertical lines across subplots to show the average %LoC corresponding to $k = 10$ on training sets. Those lines stand at 45% LoC for Impact, 42% LoC for Root Cause, 43% LoC for Attack Vector, and 44% LoC for Vulnerability Type. The differences between the best ROUGE-L scores and the scores at $k = 10$ are 0.009 (Impact), 0.002 (Root Cause), 0.005 (Attack Vector), and 0.02 (Vulnerability Type). Therefore, setting $k = 10$ is small enough to achieve the optimal performance.

Using only a few lines and all lines in the code function may not achieve the best ROUGE score. Our experiment suggests that $k = 10$ is an optimal choice.

5. Qualitative analysis

In this section, we present our qualitative analysis of the performance of our approach, which allows us to understand its strengths and limitations. Firstly, we select two examples from test sets, CVE-2019-14934,[15] and CVE-2017-10911,[16] described in Listing 2 and Listing 3, respectively. CVE-2019-14934 was selected because it performs well on

[15] https://nvd.nist.gov/vuln/detail/CVE-2019-14934
[16] https://nvd.nist.gov/vuln/detail/CVE-2017-10911

Table 9
We report ROUGE results to compare CodeT5+-200M, BART-CodeBERT, RAG (CodeT5+), RAG (BM25), and few-shot learning. R-1, R-2, R-L stand for ROUGE-1, ROUGE-2, ROUGE-L respectively.

	Impact			Root cause			Attack vector			Vulnerability type		
	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L
BART-CodeBERT	0.682	0.637	0.676	0.548	0.514	0.547	0.733	0.672	0.732	0.624	0.599	0.625
RAG (CodeT5+)	0.054	0.005	0.049	0.579	0.552	0.578	0.698	0.640	0.697	0.093	0.063	0.094
RAG (BM25)	0.056	0.005	0.052	0.506	0.478	0.505	0.631	0.562	0.630	0.094	0.062	0.094
Few-shot Learning	0.183	0.095	0.166	0.185	0.112	0.181	0.227	0.141	0.225	0.116	0.082	0.114
CodeT5+-220M	**0.897**	**0.847**	**0.896**	**0.815**	**0.748**	**0.813**	**0.851**	**0.774**	**0.850**	**0.776**	**0.641**	**0.775**

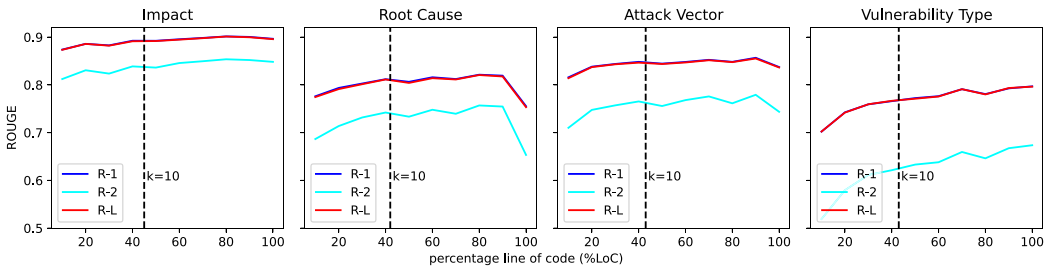

Fig. 8. ROUGE score varying against the percentage of top lines of code.

Table 10
We perform hypothesis testing to compare ROUGE scores, CodeT5+-200M against baselines. R-1, R-2, R-L stand for ROUGE-1, ROUGE-2, ROUGE-L respectively. Because there is no case that we cannot reject $H0$, X symbol is not used in this table.

	Ours			
	R-1	R-2	R-L	
BART-CodeBERT	0.697	0.596	0.707	Impact
	0.703	0.546	0.697	Root Cause
	0.386	0.089	0.386	Attack Vector
	0.386	0.089	0.386	Vulnerability Type
RAG (CodeT5+)	6.261	4.961	6.333	Impact
	0.617	0.458	0.613	Root Cause
	0.456	0.329	0.455	Attack Vector
	2.606	1.673	2.606	Vulnerability Type
RAG (BM25)	6.215	4.963	6.282	Impact
	0.799	0.625	0.793	Root Cause
	0.641	0.511	0.639	Attack Vector
	2.618	1.685	2.619	Vulnerability Type
Few-shot Learning	3.336	3.071	3.433	Impact
	2.287	1.984	2.290	Root Cause
	2.219	1.871	2.218	Attack Vector
	2.301	1.548	2.301	Vulnerability Type

the vulnerability type, while CVE-2017-10911 was chosen due to its poor performance on the impact aspect.

CVE-2019-14934 is a vulnerable code function because the function **write_version** uses the length of file name (**fname**) and the length of folder name (**dirname**) to allocate memory for **new_frame** variable (line #8). As both **fname** and **dirname** are passed from outside, attackers can leverage the logic with a lengthy file and folder name, which may lead to a malloc failure. The solution[17] for CVE-2019-14934 involves adding logic to **write_version** to ensure that the allocated memory fits the machine's capacity. In contrast, CVE-2017-10911 unintentionally leaks data when the function **make_response** processes response data. Specifically, **resp** is intended to hold data after calling **RING_GET_RESPONSE**, but in line #3, **resp** is a local structure instance on the stack. In line #15, memory on the stack is copied to it via **memcpy**. This method of populating data is vulnerable; therefore, the solution[18] for CVE-2017-10911 is to change **resp** to a pointer and explicitly fill in the necessary data fields.

As shown in Section 4, key aspect generators are trained with CodeT5+-220M pre-train which yields the best ROUGE score; hence, we use them to generate key aspects for each example. We report on their results in Table 15. Our observation is that the key-aspect explanations are relevant to the cause of vulnerability in some aspects. In CVE-2019-14934, the root cause aspect is correct, as we mentioned, **write_version** needs a validation on the size of requested memory. Additionally, CVE-2019-14934 is listed with CWE-787,[19] which is also the model output for the vulnerability type. In CVE-2017-10911, our model successfully mentions 'obtain sensitive information' in impact aspect.

In the next step, we are interested in cases where our key-aspect generators do not work well. In CVE-2017-10911, as explained, a vulnerable code occurs in vulnerable declaration at line #3, but the output mentions the failure to verify the sanity of the **ring** pointer. This phenomenal suggests that our root cause model focuses on line #11, **blk_rings** is an attribute of **ring**. Because **ring** is fed from outside, it is unclear that **ring→blk_rings** has value. In case **ring→blk_rings** return null, then **RING_GET_RESPONSE** at the line #15 would be crashed. Therefore, from this point of view, the generated root cause is still relevant. Using the phenomenal, output, pointer corruption, in vulnerability type is also relevant.

We mentioned an incomplete security description issue in Section 3.2.2 and also in a study (Guo et al., 2022) in the literature where a CVE may not have all four key aspects. For example, CVE-2019-14934 lacks details on impact and attack vector, while CVE-2017-10911 does not include an attack vector from the security analyst. Due to the absence of ground truth, we manually verify those aspects based on our knowledge when reading the code function to determine

[17] https://github.com/enferex/pdfresurrect/commit/0c4120fffa3dffe97b95c486a120eded82afe8a6

[18] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=089bc0143f489bd3a4578bdff5f4ca68fb26f341

[19] https://cwe.mitre.org/data/definitions/787.html

Table 11
We report BLEU results to compare CodeT5+-200M, BART-CodeBERT, RAG (BM25), RAG(CodeT5+), and few-shot learning. B1, B2, B3, B4 stand for BLEU-1, BLEU-2, BLEU-3, BLEU-4 respectively.

	Impact				Root cause				Attack vector				Vulnerability type			
	B1	B2	B3	B4	B1	B2	B3	B4	B1	B2	B3	B4	B1	B2	B3	B4
BART-CodeBERT	0.693	0.647	0.631	0.616	0.526	0.483	0.463	0.448	0.717	0.651	0.637	0.626	0.626	0.607	0.635	0.653
RAG (CodeT5+)	0.065	0.006	0.005	0.005	0.551	0.515	0.497	0.482	0.665	0.599	0.581	0.565	0.039	0.019	0	0
RAG (BM25)	0.068	0.006	0.005	0.005	0.469	0.433	0.418	0.402	0.596	0.518	0.499	0.483	0.04	0.019	0	0
Few-shot Learning	0.173	0.084	0.07	0.06	0.14	0.068	0.044	0.033	0.205	0.094	0.078	0.066	0.081	0.044	0.003	0.002
CodeT5+-220M	**0.916**	**0.876**	**0.842**	**0.812**	**0.82**	**0.759**	**0.723**	**0.705**	**0.852**	**0.782**	**0.747**	**0.722**	**0.768**	**0.653**	**0.693**	**0.852**

Table 12
We perform hypothesis testing to compare ROUGE scores, CodeT5+-200M against baselines. B1, B2, B3, B4 stand for BLEU-1, BLEU-2, BLEU-3, BLEU-4 respectively. We also mark 'non-normality' in cases they are cannot be concluded as normality under Shapiro–Wilk test. Then effect is derived from Vargha and Delaney method.

	Ours				
	B1	B2	B3	B4	
BART-CodeBERT	0.670	0.595	0.465	0.328	Impact
	0.683	0.537	0.411	0.319	Root Cause
	0.395	0.298	0.215	0.151	Attack Vector
	0.334	0.015	X	0.033	Vulnerability Type
RAG (CodeT5+)	5.912	4.974	3.766	2.635	Impact
	0.604	0.451	0.322	0.220	Root Cause
	0.481	0.368	0.279	0.211	Attack Vector
	2.671	1.562	0.420	**0.080 non-normality**	Vulnerability Type
RAG (BM25)	5.897	5.009	3.779	2.641	Impact
	0.782	0.617	0.467	0.340	Root Cause
	0.666	0.555	0.463	0.387	Attack Vector
	2.693	1.574	0.424	**0.079 non-normality**	Vulnerability Type
Few-shot Learning	3.352	3.102	2.624	2.046	Impact
	2.289	2.341	2.007	1.680	Root Cause
	2.237	1.929	1.688	1.513	Attack Vector
	2.154	1.371	0.545	0.403	Vulnerability Type

Table 13
We report F1-score from BERTScore for CodeT5+-200M, BART-CodeBERT, RAG (BM25), RAG(CodeT5+), and few-shot learning.

	Impact	Root cause	Attack vector	Vulnerability type
BART-CodeBERT	0.944	0.931	0.954	0.943
RAG (CodeT5+)	0.846	0.935	0.949	0.854
RAG (BM25)	0.847	0.923	0.937	0.853
Few-shot Learning	0.869	0.877	0.876	0.872
CodeT5+-220M	**0.982**	**0.967**	**0.965**	**0.979**

Table 14
We perform hypothesis testing to compare F1-score from BERTScore, CodeT5+-200M against baselines. Because there is no case that we cannot reject $H0$, X symbol is not used in this table.

	Ours	
BART-CodeBERT	0.695	Impact
	0.578	Root Cause
	0.195	Attack Vector
	0.619	Vulnerability Type
RAG (CodeT5+)	4.908	Impact
	0.507	Root Cause
	0.274	Attack Vector
	3.361	Vulnerability Type
RAG (BM25)	4.870	Impact
	0.679	Root Cause
	0.471	Attack Vector
	3.405	Vulnerability Type
Few-shot Learning	3.063	Impact
	1.864	Root Cause
	1.882	Attack Vector
	2.461	Vulnerability Type

whether a generated explanation is relevant to the missing aspect. In CVE-2019-14934, the **write_version** function has an out-of-bound weakness caused by external control through its arguments. An attacker could logically exploit this to cause a crash in the application. According to OWASP,[20] out-of-bound conditions are one of the methods through which a denial-of-service attack can target a resource to make it unavailable. Therefore, the impact generated for CVE-2019-14934 is relevant. Regarding the attack vector aspect for both cases, we do not have enough context to judge relevancy; however, the generated explanation mentions "file" for CVE-2019-14934 and "handling of input" for CVE-2017-10911.

```
static void write_version(FILE *fp, const char
*fname, const char *dirname,
xref_t *xref) {
  long start;
  char *c, *new_fname, data;
  FILE *new_fp;
  start = ftell(fp);
  if((c = strstr(fname, ".pdf"))) c = '\0';
  new_fname
  = malloc(strlen(fname) + strlen(dirname) + 16);
  // vulnerable statement
  snprintf(new_fname,      strlen(fname)      +
  strlen(dirname) + 16,
     "%s%s-version-%d.pdf",   dirname,   fname,
  xref->version);
  // ...
}
```

Listing 2: CVE-2019-14934.

```
static void make_response(struct xen_blkif_ring
*ring, u64 id,
unsigned short op, int st) {
  struct blkif_response resp; // vulnerable
      statement
  unsigned long flags;
  union blkif_back_rings *blk_rings;
  int notify;
  resp.id = id; // vulnerable statement
  resp.operation = op; // vulnerable
      statement
  resp.status = st; // vulnerable statement
  spin_lock_irqsave(&ring->blk_ring_lock,
      flags);
  blk_rings = &ring->blk_rings;
  switch (ring->blkif->blk_protocol) {
    case BLKIF_PROTOCOL_NATIVE:
      memcpy(
        RING_GET_RESPONSE(&blk_rings->native,
blk_rings->native.rsp_prod_pvt),
&resp, sizeof(resp)); // vulnerable statement
      break;
    // ...
}
```

Listing 3: CVE-2017-10911.

6. User study

To investigate how experienced software practitioners evaluate our vulnerability explanations in a pre-merge code review context, we conducted a small-scale study.[21] We targeted practitioners with over two years of C/C++ development experience and a demonstrated interest in vulnerability prevention. Recognizing the significant time commitment required from such knowledgeable participants, we opted for a sample size of 6–10 individuals, a practice consistent with established software engineering research employing similar methodologies (Pan et al., 2023a; Sun et al., 2023; Zhang et al., 2023). The population of participants we recruited from was diverse, including software engineers and security experts with different experience and background.

Our Ethics Procedure. We went through an ethics approval process at the University of Wollongong. All text is required to show as its approved version to participants. When participants agree, their privacy protection measures and benefits are:

1. That their contribution will be confidential and that there will be no personal identification in the data that they agree to allow to be used in the study
2. That there are no potential risks associated with this study
3. That their participation in this research is voluntary and no financial incentives
4. If any concerns or complaints regarding the way the research is or has been conducted, they can contact the University of Wollongong Ethics Officer

Recruiting Advertisement. We posted our call for participants, who are enthusiastic about vulnerability prevention, in multiple channels. First, we directly contacted our colleagues and friends working directly in the software industry. Second, we employed LinkedIn to identify technical leaders and senior software engineers who worked in security domains and participated in Pwn2Own competitions.[22] Then, we asked

[20] https://owasp.org/www-community/attacks/Denial_of_Service

[21] Ethics approval #2024/102 from University of Wollongong.

[22] https://en.wikipedia.org/wiki/Pwn2Own

Table 15
Results generated key aspects for CVE-2019-14934, and CVE-2017-10911 by our approach and security experts.

	CVE-2017-10911	CVE-2019-14934	
Root Cause	The failure to sanity check the ringproducer/consumer pointers	Does not validate a certain size value	Ours
Impact	Leak kernel memory bytesand obtain sensitive information	Execute arbitrary code or cause adenial of service condition	
Attack Vector	Via vectors related to the handling of input	Persuading a victimto open a specially-crafted file	
Vulnerability Type	Pointer corruption	Out-of-bounds write	
Root Cause	Leveraging the copying of uninitialized padding fields	Does not validate a certain size value	Experts
Impact	Obtain sensitive information from host OS	N/A	
Attack Vector	N/A	N/A	
Vulnerability Type	Exposure of sensitive information	Out-of-bounds write	

for their help to share our call for participants with their colleagues or friends. Lastly, we circulated our call for participants on social networks like LinkedIn, Facebook, and X (former Twitter). The social platforms' statistics show that our advertisement achieved over 1,500 impressions.

Participants. We recruited ten software practitioners with over two years of experience in the IT industry, foundational knowledge of C/C++ programming languages, and proficiency in reading and writing code and technical documents in English. The participants were from companies in six various countries, such as Australia, Singapore, Vietnam, Finland, South Korea, and the USA.

Task Description. We randomly selected five code functions from our test set. For each sample, we showed the top-10 predicted lines and also the four key aspects, and asked the participants to answer the following questions:

1. Based on the above explanation, do you accept the prediction that the function is vulnerable? (Yes/No question)
2. If you accept the prediction, how useful are the key aspects in helping you decide whether to accept or reject the vulnerability prediction?
3. Please rank the usefulness of the four aspects when accepting the prediction.
4. If you chose to accept the prediction, how useful are the key aspects in helping you debug and fix the vulnerability?
5. Please rank the usefulness of the four aspects in supporting you in debugging and fixing the vulnerability.

Questions #1, #2, and #3 evaluate how key aspects influence participants' decisions to accept or reject the vulnerability prediction. On the other hand, questions #4 and #5 assess how key aspects assist participants in debugging and fixing the vulnerability. Questions #2 and #4 are 5-point Likert scale questions that ask participants how the usefulness of key aspects influences their task. Additionally, Questions #3 and #5 ask participants to rank the usefulness of all four key aspects in supporting their corresponding tasks. In our survey, participants assign a number from 1 to 4 to each key aspect, the higher the more useful. For example, if a participant finds Impact more important than Root Cause, they will assign a higher score to Impact than to Root Cause.

We aggregated all responses for each question across five code functions and calculated the percentage of responses for each option. We have found that 72% responded "Yes" to the first question, which suggests that the provided explanation (located lines of code and human-readable aspects) helped them accept the prediction. For Question #2, a majority (68%) responded that the aspects were "moderately useful", "very useful", or even "extremely useful", which together account for 68% of the total responses. 22% of responses were "slightly useful" while only 10% found the aspects "not at all useful". Regarding Question #3, where participants assigned a number from 1 to 4 to each key aspect, we calculated the average score for each aspect. The results showed that Attack Vector was rated the most useful (average score of 3.14), followed by Vulnerability Type (2.8), Impact (2.22), and Root Cause (1.84).

Regarding the answers to questions #4 and #5, 86.1% of the responses indicated that the key aspects were very useful or extremely useful for debugging and fixing the vulnerability. In addition, Attack Vector was rated the most useful aspect (average score of 3.14), followed by Vulnerability Type (2.72), Impact (2.39), and Root Cause (1.75). Overall, results from this user study demonstrate the usefulness of our approach in helping software developers not only trust the prediction of a vulnerability in the code but also debug and fixing.

7. Threats to validity

Internal validity. Threats to internal validity refer to errors in our experiments. Due to our limitations on GPU resources, we reduce floating point precision to 16-bit. Next, we employed code published in paper (Sun et al., 2023) for RQ2 and closely followed the description of their work; however, errors may remain. Yet there is a lack of ablation analysis to assess the contribution of pre-training the model. Although our framework is inherently designed to leverage the pre-training, without the ablation, we are unable to rule out the possibility that similar results could be achieved when training the model from scratch. Regarding the user study, the number of participants in our user study is small. Additionally, we are unable to analyze participants' backgrounds, such as their company, job title, and years of experience. Thus, the study may not be representative of the entire software development community. Finally, the authors will update the pre-trained models we used. Therefore, model version mismatch may lead to different results.

External validity. We assess the quality of our aspect generation based on positive samples. Although our models exhibit very high ROUGE scores, positive samples in the vulnerability domain remain rare, and the impact of noise on unseen data requires further investigation. Additionally, our study does not account for the effect of incorrect vulnerability predictions, which may result in our framework providing irrelevant explanations. Moreover, the potential reasons for misleading explanations and the impact of incorrect explanations are not examined. Consequently, there is a lack of risk management tools to facilitate a deployment of our framework

8. Related work

Current advancements in software engineering have seen a surge in interest in language models (LM) and transformer architectures (Vaswani et al., 2017). Those models have been applied to different software engineering tasks such as code completion (Lu et al., 2021; Liu et al., 2021), code summarization (Ahmad et al., 2020; Zan et al., 2023), code translation (Pan et al., 2023b; Chen et al., 2021), clone detection (White et al., 2016), and code testing (Zheng et al., 2023).

In SVP, LMs are often employed for source code embedding, which involves converting textual tokens into vector representations. Natural language understanding techniques such as Doc2Vec (Le and Mikolov, 2014), GloVe (Pennington et al., 2014), and Word2Vec (Mikolov et al., 2013) have been frequently used to generate vectors for each token, as observed in studies (Li et al., 2021; Chakraborty et al., 2022; Cheng et al., 2021). Advanced models, including CodeBERT (Feng et al., 2020)

and GraphCodeBERT (Guo et al., 2021), offer superior embeddings for code functions.

Regarding explainable artificial intelligence (XAI) approaches, the topic is categorized into instance-level and model-level methods (Yuan et al., 2023). Instance-level methods provide explanations for individual inputs by pinpointing the crucial input features that influence a DL model's prediction. By contrast, model-level approaches (Erhan et al., 2009; Yuan et al., 2020) delve into the foundational patterns and attributes within datasets that inform the AI model's decision-making processes.

Recent XAI research on vulnerability predictions have been focusing on providing explanations under instance-level. For instance, Li et al. (2021), Nguyen et al. (2025) pioneered a graph-based method that generates explanations through subgraphs, which encapsulate code statements pertinent to identified vulnerabilities within a specific source code. Fu and Tantithamthavorn (2022) utilized attention mechanisms within their AI model to attribute weights to tokens, thus facilitating the identification of fine-grained locations of vulnerabilities in a given source code. However, these studies have not sufficiently provided explanations in human language, or a generalized information, which are essential to fostering human interpretation.

Providing human-understandable explanations has been a topic of interest in other applications. Salam et al. (2022) delved into techniques that utilize learned rules to articulate image predictions in natural language. Similarly, Zhou et al. (2020) devised a framework that distills the reasoning of neural models into coherent natural language explanations, showcasing its utility in sentiment analysis tasks.

9. Conclusions and future work

In this paper, we presented a framework which can assist software practitioners to verify results from SVP and fix security issues during the software development phase. Our framework, which is based on the Seq2seq transformer architecture, can be integrated with any vulnerability prediction methods and provides explanations of four key aspects for a given vulnerability: Attack Vector, Root Cause, Impact, and Vulnerability Type. We conducted a comprehensive empirical study to evaluate the quality of our proposed framework using various pre-trained language models. In future work, we will investigate whether using a single model to generate all key aspects. We also plan to address limitations in our current framework, such as the lack of ablation analysis to assess the contribution of pre-training the model, and conduct a large-scale user study. In addition, we are interested in the development of our framework for developers that can be integrated into their IDEs or CI/CD pipelines.

CRediT authorship contribution statement

Hong Quy Nguyen: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation, Conceptualization. **Thong Hoang:** Writing – review & editing, Supervision. **Hoa Khanh Dam:** Writing – review & editing, Supervision. **Guoxin Su:** Writing – review & editing. **Zhenchang Xing:** Writing – review & editing. **Qinghua Lu:** Writing – review & editing. **Jiamou Sun:** Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We share code on the link, described in the paper. We cited the paper from which the data originated.

References

Achtibat, R., Dreyer, M., Eisenbraun, I., Bosse, S., Wiegand, T., Samek, W., Lapuschkin, S., 2022. From "where" to "what": Towards human-understandable explanations through concept relevance propagation. arXiv:2206.03208.

Aggarwal, A., Jalote, P., 2006. Integrating static and dynamic analysis for detecting vulnerabilities. In: 30th Annual International Computer Software and Applications Conference. COMPSAC'06, vol. 1, IEEE, pp. 343–350.

Ahmad, W., Chakraborty, S., Ray, B., Chang, K.-W., 2020. A transformer-based approach for source code summarization. In: ACL (Short).

Aloraini, B., Nagappan, M., German, D.M., Hayashi, S., Higo, Y., 2019. An empirical study of security warnings from static application security testing tools. J. Syst. Softw. 158, 110427. http://dx.doi.org/10.1016/j.jss.2019.110427, URL https://www.sciencedirect.com/science/article/pii/S0164121219302018.

Anon, 2018. Cve program request forms, key details phrasing. URL https://www.cve.org/Resources/General/Key-Details-Phrasing.pdf.

Anon, 2022a. National vulnerability database. URL https://nvd.nist.gov.

Anon, 2022b. IBM X-Force. URL https://exchange.xforce.ibmcloud.com/activity/list?filter=Vulnerabilities.

Anon, 2023. Github code dataset. URL https://huggingface.co/datasets/codeparrot/github-code.

Avgerinos, T., Cha, S.K., Rebert, A., Schwartz, E.J., Woo, M., Brumley, D., 2014. Automatic exploit generation. Commun. ACM 57 (2), 74–84. http://dx.doi.org/10.1145/2560217.2560219.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20, Curran Associates Inc., Red Hook, NY, USA.

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9), 3280–3296. http://dx.doi.org/10.1109/TSE.2021.3087402.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y., 2021. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. ACM Trans. Softw. Eng. Methodol. 30 (3), http://dx.doi.org/10.1145/3436877.

Dam, H.K., Tran, T., Ghose, A., 2018b. Explainable software analytics. In: Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results. In: ICSE-NIER '18, Association for Computing Machinery, New York, NY, USA, pp. 53–56. http://dx.doi.org/10.1145/3183399.3183424.

Dam, H.K., Tran, T., Pham, T., Ng, S.W., Grundy, J., Ghose, A., 2018a. Automatic feature learning for predicting vulnerable software components. IEEE Trans. Softw. Eng. 47 (1), 67–85. http://dx.doi.org/10.1109/TSE.2018.2881961.

Devlin, J., Chang, M., Lee, K., Toutanova, K., 2019. BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (Eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, pp. 4171–4186. http://dx.doi.org/10.18653/V1/N19-1423.

Erhan, D., Bengio, Y., Courville, A., Vincent, P., 2009. Visualizing higher-layer features of a deep network. Univ. Montr. 1341 (3), 1.

Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In: 2020 IEEE/ACM 17th International Conference on Mining Software Repositories. MSR, pp. 508–512. http://dx.doi.org/10.1145/3379597.3387501.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020. Association for Computational Linguistics, Online, pp. 1536–1547. http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139, URL https://aclanthology.org/2020.findings-emnlp.139.

Fu, M., Tantithamthavorn, C., 2022. LineVul: A transformer-based line-level vulnerability prediction. In: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories. MSR, pp. 608–620. http://dx.doi.org/10.1145/3524842.3528452.

Ghaffarian, S.M., Shahriari, H.R., 2017. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. ACM Comput. Surv. 50 (4), 1–36.

Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep Learning. MIT Press, http://www.deeplearningbook.org.

Guo, H., Chen, S., Xing, Z., Li, X., Bai, Y., Sun, J., 2022. Detecting and augmenting missing key aspects in vulnerability descriptions. ACM Trans. Softw. Eng. Methodol. 31 (3), http://dx.doi.org/10.1145/3498537.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., LIU, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M., 2021. GraphCode{BERT}: Pre-training code representations with data flow. In: International Conference on Learning Representations. URL https://openreview.net/forum?id=jLoC4ez43PZ.

Hin, D., Kan, A., Chen, H., Babar, M.A., 2022. LineVD: Statement-level vulnerability detection using graph neural networks. In: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories. MSR, pp. 596–607. http://dx.doi.org/10.1145/3524842.3527949.

Hoffman, R.R., Mueller, S.T., Klein, G., Litman, J., 2018. Metrics for explainable AI: Challenges and prospects. arXiv preprint arXiv:1812.04608.

Hunter, T., Vynck, G., 2021. The 'most serious' security breach ever is unfolding right now. Here's what you need to know. Wash. Post.

James Newsome, D.S., 2005. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. Proc. Netw. Distrib. Syst. Secur. Symp..

Jibilian, I., Canales, K., 2021. The US is readying sanctions against Russia over the SolarWinds cyber attack. Here'sa simple explanation of how the massive hack happened and why it's such a big deal. Bus. Insid..

Kim, S., Kim, R.Y.C., Park, Y.B., 2016. Software vulnerability detection methodology combined with static and dynamic analysis. Wirel. Pers. Commun. 89, 777–793.

Krsul, I.V., 1998. Software Vulnerability Analysis. Purdue University.

Kuang, C., Miao, Q., Chen, H., 2006. Analysis of software vulnerability. In: Proceedings of the 5th WSEAS International Conference on Information Security and Privacy. pp. 218–223.

Le, Q., Mikolov, T., 2014. Distributed representations of sentences and documents. In: Xing, E.P., Jebara, T. (Eds.), Proceedings of the 31st International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 32, (2), PMLR, Bejing, China, pp. 1188–1196, URL https://proceedings.mlr.press/v32/le14.html.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., Zettlemoyer, L., 2020b. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, Online, pp. 7871–7880. http://dx.doi.org/10.18653/v1/2020.acl-main.703, URL https://aclanthology.org/2020.acl-main.703.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D., 2020a. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems. vol. 33, Curran Associates, Inc., pp. 9459–9474, URL https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

Li, Y., Wang, S., Nguyen, T.N., 2021. Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. In: ESEC/FSE 2021, Association for Computing Machinery, New York, NY, USA, pp. 292–303. http://dx.doi.org/10.1145/3468264.3468597.

Lin, C.-Y., 2004. ROUGE: A package for automatic evaluation of summaries. In: Text Summarization Branches Out. Association for Computational Linguistics, Barcelona, Spain, pp. 74–81, URL https://aclanthology.org/W04-1013.

Liu, F., Li, G., Zhao, Y., Jin, Z., 2021. Multi-task learning based pre-trained language model for code completion. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. ASE '20, Association for Computing Machinery, New York, NY, USA, pp. 473–485. http://dx.doi.org/10.1145/3324884.3416591.

Liu, B., Shi, L., Cai, Z., Li, M., 2012. Software vulnerability discovery techniques: A survey. In: 2012 Fourth International Conference on Multimedia Information Networking and Security. IEEE, pp. 152–156.

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C.B., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S.K., Fu, S., Liu, S., 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. CoRR, arXiv:2102.04664.

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H., 2018. Mixed precision training. arXiv:1710.03740.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. In: Burges, C., Bottou, L., Welling, M., Ghahramani, Z., Weinberger, K. (Eds.), Advances in Neural Information Processing Systems. vol. 26, Curran Associates, Inc., URL https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.

Muller, K., 1989. Statistical Power Analysis for the Behavioral Sciences. Taylor & Francis.

Nashid, N., Sintaha, M., Mesbah, A., 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning . In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE Computer Society, pp. 2450–2462, URL https://doi.ieeecomputersociety.org/10.1109/ICSE48619.2023.00205.

Nguyen, H.Q., Hoang, T., Dam, H.K., Ghose, A., 2025. Graph-based explainable vulnerability prediction. Inf. Softw. Technol. 177, 107566. http://dx.doi.org/10.1016/j.infsof.2024.107566.

Pan, S., Bao, L., Xia, X., Lo, D., Li, S., 2023a. Fine-grained commit-level vulnerability type prediction by CWE tree structure. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, pp. 957–969. http://dx.doi.org/10.1109/ICSE48619.2023.00088.

Pan, R., Ibrahimzada, A.R., Krishna, R., Sankar, D., Wassi, L.P., Merler, M., Sobolev, B., Pavuluri, R., Sinha, S., Jabbarvand, R., 2023b. Understanding the effectiveness of large language models in code translation. arXiv preprint arXiv:2308.03109.

Papineni, K., Roukos, S., Ward, T., Zhu, W.-J., 2002. BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting on Association for Computational Linguistics. ACL '02, Association for Computational Linguistics, USA, pp. 311–318. http://dx.doi.org/10.3115/1073083.1073135.

Parnin, C., Orso, A., 2011. Are automated debugging techniques actually helping programmers? In: Proceedings of the 2011 International Symposium on Software Testing and Analysis. ISSTA '11, Association for Computing Machinery, New York, NY, USA, pp. 199–209. http://dx.doi.org/10.1145/2001420.2001445.

Pennington, J., Socher, R., Manning, C., 2014. GloVe: Global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, Doha, Qatar, pp. 1532–1543. http://dx.doi.org/10.3115/v1/D14-1162, URL https://aclanthology.org/D14-1162.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., 2019. Language models are unsupervised multitask learners. URL https://api.semanticscholar.org/CorpusID:160025533.

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications. ICMLA, pp. 757–762. http://dx.doi.org/10.1109/ICMLA.2018.00120.

Salam, A., Schwitter, R., Orgun, M.A., 2022. HESIP: A hybrid system for explaining sub-symbolic predictions. In: AI 2021: Advances in Artificial Intelligence: 34th Australasian Joint Conference, AI 2021, Sydney, NSW, Australia, February 2–4, 2022, Proceedings. Springer-Verlag, Berlin, Heidelberg, pp. 27–39. http://dx.doi.org/10.1007/978-3-030-97546-3_3.

Sampaio, L., Garcia, A., 2016. Exploring context-sensitive data flow analysis for early vulnerability detection. J. Syst. Softw. 113, 337–361.

Sennrich, R., Haddow, B., Birch, A., 2016. Neural machine translation of rare words with subword units. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Berlin, Germany, pp. 1715–1725. http://dx.doi.org/10.18653/v1/P16-1162, URL https://aclanthology.org/P16-1162.

Shah, S., Mehtre, B.M., 2015. An overview of vulnerability assessment and penetration testing techniques. J. Comput. Virol. Hacking Tech. 11, 27–49.

Shapiro, S.S., Wilk, M.B., 1965. An analysis of variance test for normality (complete samples). Biometrika 52 (3/4), 591–611, URL http://www.jstor.org/stable/2333709.

Smith, J., Johnson, B., Murphy-Hill, E., Chu, B., Lipford, H.R., 2015. Questions developers ask while diagnosing potential security vulnerabilities with static analysis. In: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. In: ESEC/FSE 2015, Association for Computing Machinery, New York, NY, USA, pp. 248–259. http://dx.doi.org/10.1145/2786805.2786812.

Sparks, S., Embleton, S., Cunningham, R., Zou, C., 2007. Automated vulnerability analysis: Leveraging control flow for evolutionary input crafting. In: Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). IEEE, pp. 477–486.

Sun, J., Xing, Z., Lu, Q., Xu, X., Zhu, L., Hoang, T., Zhao, D., 2023. Silent vulnerable dependency alert prediction with vulnerability key aspect explanation. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, pp. 970–982. http://dx.doi.org/10.1109/ICSE48619.2023.00089.

Sutton, M., Greene, A., Amini, P., 2007. Fuzzing: Brute Force Vulnerability Discovery. Pearson Education.

Tillmann, C., Ney, H., 2003. Word Reordering and a Dynamic Programming Beam Search Algorithm for Statistical Machine Translation. Comput. Linguist. 29 (1), 97–133. http://dx.doi.org/10.1162/089120103321337458, arXiv:https://direct.mit.edu/coli/article-pdf/29/1/97/1797924/089120103321337458.pdf.

Vargha, A., Delaney, H.D., 2000. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. J. Educ. Behav. Stat. 25 (2), 101–132.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L.u., Polosukhin, I., 2017. Attention is all you need. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems. vol. 30, Curran Associates, Inc., URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Wang, Y., Bashar, M.A., Chandramohan, M., Nayak, R., 2023a. Exploring topic models to discern cyber threats on Twitter: A case study on Log4Shell. Intell. Syst. Appl. 20, 200280. http://dx.doi.org/10.1016/j.iswa.2023.200280, URL https://www.sciencedirect.com/science/article/pii/S2667305323001059.

Wang, Y., Le, H., Gotmare, A.D., Bui, N.D., Li, J., Hoi, S.C.H., 2023c. CodeT5+: Open code large language models for code understanding and generation. ArXiv Preprint.

Wang, W., Nguyen, T.N., Wang, S., Li, Y., Zhang, J., Yadavally, A., 2023b. DeepVD: Toward class-separation features for neural network vulnerability detection. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, pp. 2249–2261. http://dx.doi.org/10.1109/ICSE48619.2023.00189.

White, M., Tufano, M., Vendome, C., Poshyvanyk, D., 2016. Deep learning code fragments for code clone detection. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. pp. 87–98.

Yuan, H., Tang, J., Hu, X., Ji, S., 2020. Xgnn: Towards model-level explanations of graph neural networks. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 430–438.

Yuan, H., Yu, H., Gui, S., Ji, S., 2023. Explainability in graph neural networks: A taxonomic survey. IEEE Trans. Pattern Anal. Mach. Intell. 45 (5), 5782–5799. http://dx.doi.org/10.1109/TPAMI.2022.3204236.

Yuen, K.K., Dixon, W.J., 1973. The approximate behaviour and performance of the two-sample trimmed t. Biometrika 60 (2), 369–374, URL http://www.jstor.org/stable/2334550.

Zan, D., Chen, B., Zhang, F., Lu, D., Wu, B., Guan, B., Yongji, W., Lou, J.-G., 2023. Large language models meet NL2code: A survey. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 7443–7464.

Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y., 2020. BERTScore: Evaluating text generation with BERT. In: International Conference on Learning Representations. URL https://openreview.net/forum?id=SkeHuCVFDr.

Zhang, J., Liu, S., Wang, X., Li, T., Liu, Y., 2023. Learning to locate and describe vulnerabilities. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering. ASE, pp. 332–344. http://dx.doi.org/10.1109/ASE56229.2023.00045.

Zheng, Z., Ning, K., Chen, J., Wang, Y., Chen, W., Guo, L., Wang, W., 2023. Towards an understanding of large language models in software engineering tasks. arXiv preprint arXiv:2308.11396.

Zhou, S., Alon, U., Agarwal, S., Neubig, G., 2023. CodeBERTScore: Evaluating code generation with pretrained models of code. In: Bouamor, H., Pino, J., Bali, K. (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Singapore, pp. 13921–13937. http://dx.doi.org/10.18653/v1/2023.emnlp-main.859, URL https://aclanthology.org/2023.emnlp-main.859.

Zhou, W., Hu, J., Zhang, H., Liang, X., Sun, M., Xiong, C., Tang, J., 2020. Towards interpretable natural language understanding with explanations as latent variables. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20, Curran Associates Inc., Red Hook, NY, USA.